HireReady — Study Guide for Interviews

This document explains the application end-to-end: what it does, how it's built, how RAG fits in, and how every piece connects. Updated as features are built.

What is HireReady?

HireReady is a full-stack AI tool that helps job seekers in four steps:

1. **Upload a resume** (PDF or DOCX)
2. **Get an ATS score** — how well the resume matches a job description
3. **See the gap** — which keywords and skills are missing
4. **Ace the interview** — AI-generated Q&A tailored to the role

The AI layer uses a technique called **RAG (Retrieval-Augmented Generation)**, explained in detail below.

The Tech Stack

Layer	Technology	Purpose
Frontend	React + TypeScript	User interface
Build tool	Vite	Fast dev server + bundler
HTTP client	axios	Frontend → backend requests
Icons	lucide-react	Consistent icon set
Backend	FastAPI (Python)	API server, business logic, file handling
Validation	Pydantic	Request/response type enforcement
File uploads	python-multipart	Parses multipart/form-data in FastAPI
AI (planned)	OpenAI / LLM	Score generation and interview coaching
AI (planned)	Vector DB	Storing resume chunks for RAG retrieval

---

How the Frontend and Backend Connect

```
Browser (React + Vite)          http://127.0.0.1:5173
         │
         │  HTTP requests (axios)
         │
FastAPI backend (uvicorn)       http://127.0.0.1:8000
         │
         ├── GET  /api/health           → health check
         └── POST /api/resume/upload    → file upload
```

They are two separate processes. The frontend talks to the backend over HTTP. **CORS** (Cross-Origin Resource Sharing) is configured in FastAPI to allow requests from the frontend's origin (`127.0.0.1:5173` and `5174`).

Why separate processes? In production the frontend becomes static files on a CDN (Vercel, Cloudflare). The backend runs on a server (Railway, AWS). They talk over HTTPS. Keeping them separate from day one mirrors that setup exactly.

Folder Structure and Why It's Organised This Way

```
backend/
  app/
    api/
      routes.py       ← Router that wires all sub-routers together
      resume.py       ← Resume route handlers (thin — no logic)
    services/
      resume_service.py  ← All business logic for resumes
    models/
      resume.py       ← Pydantic request/response models
    core/
      config.py       ← Env vars (APP_NAME, FRONTEND_URL, etc.)
    main.py           ← FastAPI app + CORS middleware

frontend/
  src/
    components/
      ResumeUpload.tsx   ← Drag-and-drop upload widget
      HowItWorks.tsx     ← Interactive four-step tutorial section
    App.tsx              ← Root layout: nav + hero + upload card
```

Routes are kept thin. A route handler receives the request, calls a service, returns the response. It never contains business logic.

Services contain the logic. `resume_service.py` validates file type, reads and measures bytes, generates a UUID filename, writes to disk. If we later swap local disk for S3, only the service changes — the route is untouched.

Pydantic models enforce types. FastAPI uses Pydantic to auto-validate every incoming request. Wrong shape → 422 error automatically, no manual checks needed.

Current Feature: Resume Upload

End-to-end flow

1. User selects or drags a PDF/DOCX onto the upload zone
2. Frontend validates file type and size instantly (client-side, no round trip)
3. User clicks "Upload Resume"
4. Frontend sends `POST /api/resume/upload` as `multipart/form-data`
5. FastAPI receives the file as an `UploadFile` object
6. `resume_service.save_resume()` runs:
   - Checks `content_type` (PDF or DOCX only → 400 if not)
   - Reads all bytes, checks total size (max 5 MB → 400 if over)
   - Generates a UUID filename (`a3f1b2c4-...pdf`) to prevent collisions
   - Writes the file to `backend/uploads/`

7. Returns `{ filename, size, message }` as JSON
8. Frontend shows a success state with the original filename

Why UUID filenames?

If two users both upload `resume.pdf`, the second would overwrite the first. UUID generates a unique 128-bit identifier for every upload, making collisions statistically impossible.

Validation is done twice — why?

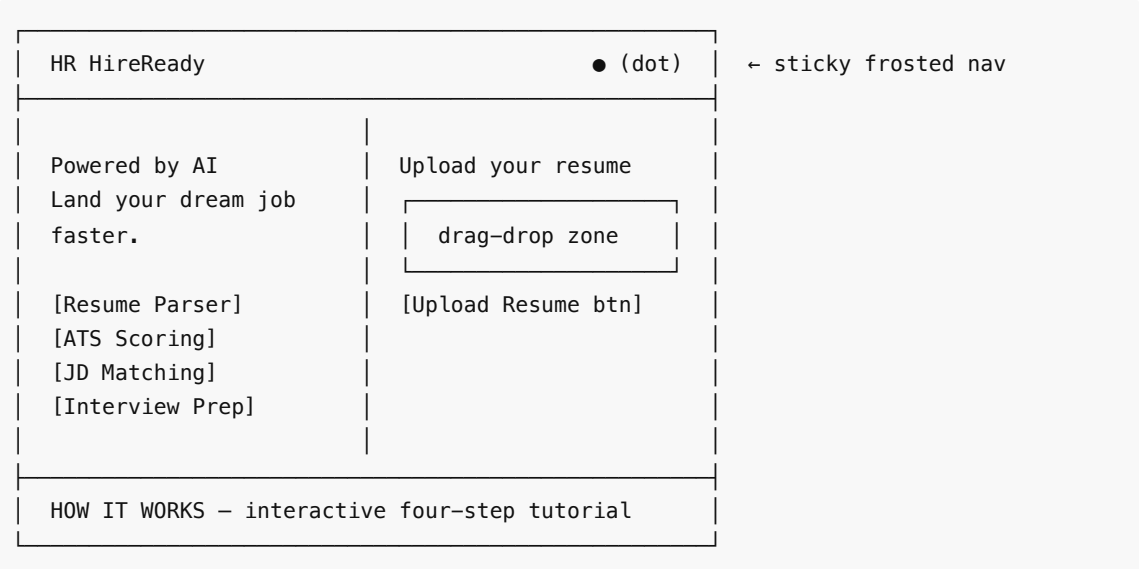
Layer	Why validate here?
Frontend	Instant feedback — user sees the error before uploading
Backend	Security — frontend checks can be bypassed with curl

Never trust the client. The backend always re-validates.

The UI

Layout

The app uses a **two-column layout** on desktop:



On mobile, columns stack vertically.

Visual techniques used

Technique	Where used	Why
Frosted glass	Navbar, upload card, how-it-works card	Depth without heavy shadows
Blob gradients	Full-page background (fixed position)	Colour without noise
Gradient text	"dream job" in hero headline	Modern emphasis
CSS keyframes	Score ring, bar fills, slide transitions	Feels alive and responsive

`requestAnimationFrame`	Score counter (0 → 78)	Smooth 60fps number count-up
Backdrop filter	Nav + cards	Glass morphism effect

How It Works — interactive section

A four-step stepper that users can click through (or use arrow keys):

Step	What it shows
1	Upload mock — drag-drop zone with a selected file preview
2	ATS Score — animated SVG ring + animated bar chart
3	Gap analysis — skill grid (green ✓ / red ✕) + missing keyword tags
4	Interview prep — real typeable textarea + reveal suggested answer

Each step slides in from the correct direction. The connector lines between step circles fill with a gradient as you progress. Keyboard left/right arrow keys navigate between steps.

What is RAG? (And How It Will Be Used Here)

RAG stands for **Retrieval-Augmented Generation**. It grounds AI answers in specific documents instead of relying on general training data.

The Problem RAG Solves

A plain LLM (like GPT-4) can discuss resumes in general, but it hasn't seen *your* resume. You could paste the whole resume into every prompt — but resumes can be long, and LLMs have context limits and cost per token.

The RAG Solution

Phase 1 — Indexing (runs once when you upload)

```
Resume text
    |
Text splitter     → breaks resume into chunks (~200 words each)
    |
Embedding model   → converts each chunk into a vector (list of numbers)
    |
Vector database   → stores vectors, indexed for fast similarity search
```

A **vector** is a list of ~1,500 numbers that encodes the *meaning* of a piece of text. Texts with similar meaning have vectors that are numerically close together (measured by cosine similarity).

Phase 2 — Retrieval + Generation (runs when you request a score)

```
Job description (the query)
    |
Embedding model       → convert JD into a vector too
    |
```

```
Vector DB search      → find resume chunks whose vectors are closest
    │                    to the JD vector (semantic similarity)
Top-k chunks          → e.g. the 3 most relevant resume sections
    │
LLM prompt:
  "Resume sections: [chunks]
   Job description: [JD]
   Score the match and list what's missing."
    │
LLM response          → ATS score + improvement suggestions
```

Why is this better than pasting the whole resume?

- Scales to very long documents
- Only the *relevant* parts go into the prompt → lower cost, better focus
- The vector DB can search across multiple resumes efficiently

Where RAG will live in HireReady

```
POST /api/resume/upload
  → save_resume()              ← built ✓
  → parse_resume()             ← next: extract text from PDF/DOCX
  → chunk_and_embed()          ← split, embed, store in vector DB

POST /api/resume/score
  → embed_job_description()
  → retrieve_relevant_chunks() ← vector DB similarity search
  → call_llm(chunks + JD)      ← generate score + suggestions
  → return ScoreResponse
```

The vector DB will likely be **ChromaDB** (local, zero infra) or **Pinecone** (cloud, scales easily). The embedding model will be OpenAI's `text-embedding-3-small`.

Interview Talking Points

"Walk me through a user request end to end."

> *The user uploads a PDF. React sends it as multipart/form-data to `POST /api/resume/upload`. FastAPI receives it as an `UploadFile`, the service layer validates content type and size, saves it with a UUID filename to disk, and returns the original filename and byte count. The frontend shows a success state. In the next phase, when the user pastes a job description, we'll parse the saved file into text, split it into chunks, embed each chunk into a vector, store those in a vector DB, then at score-time retrieve the most relevant chunks via cosine similarity against the embedded JD, and feed chunks + JD into an LLM to produce a score and a list of missing keywords.*

"Why FastAPI over Flask or Django?"

> *FastAPI gives async support out of the box (critical for slow LLM API calls), automatic request validation via Pydantic, and auto-generated OpenAPI docs at `/docs`. Django is a full framework with ORM and admin — overkill for an API-first backend. Flask is simpler but requires adding Pydantic and async support manually.*

"What is an embedding?"

> *An embedding is a vector of floating-point numbers that represents the semantic meaning of a piece of text. Two sentences that mean the same thing will have vectors that are numerically close in the high-dimensional space, even if they share no keywords. This enables semantic search — finding content by meaning rather than by exact word match.*

"How do you handle file validation security?"

> *We validate on both sides. On the frontend, the MIME type is checked before the request leaves the browser — this gives instant UX feedback. On the backend, we re-check the content type from the multipart headers and enforce a size limit by reading all bytes and measuring them server-side (not trusting the `Content-Length` header, which can be spoofed). UUID filenames prevent any path traversal or overwrite attack.*

"What is CORS and why do you need it?"

> *CORS (Cross-Origin Resource Sharing) is a browser security policy that blocks a web page from making requests to a different origin than the one it was loaded from. Our frontend is on port 5173 and backend on 8000 — different origins. FastAPI's `CORSMiddleware` adds `Access-Control-Allow-Origin` headers to responses, telling the browser it's safe to allow those cross-origin requests.*

"Why split frontend and backend?"

> *In production the frontend deploys as static files to a CDN — no server needed. The backend runs independently and can scale separately. Keeping them decoupled from the start means we never have to refactor for deployment. It also mirrors how every modern SaaS is built.*

What's Being Built Next

Feature	What it does
Resume parser	Extracts plain text from uploaded PDF / DOCX
Chunker + embedder	Splits text, generates embeddings, stores in vector DB
ATS scorer	Retrieves relevant chunks, calls LLM, returns score
JD matching	Same RAG pipeline, tuned for job description matching
Authentication	Ties uploads and scores to a user account